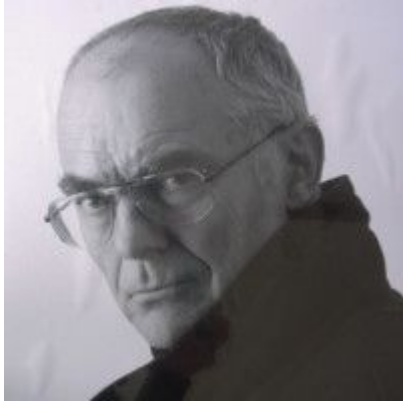


Árvore B

Leonardo da Silva Simões

Origem?

Criada por Rudolf Bayer e Edward Meyers McCreight em 1971. Não se sabe a origem do nome, pode ser uma homenagem a um dos criadores, ou uma homenagem à Boeing Scientific Research Labs, local onde a estrutura foi criada, ou pelo termo balanceamento.



Rudolf Bayer



Edward Meyers McCreight

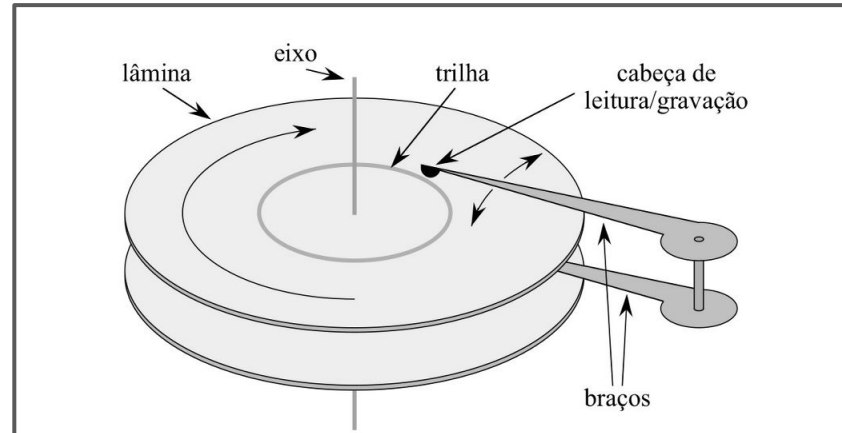
Por que a Árvore B existe?

Problema de Acessos a Discos

Acesso lento devido
a partes mecânicas

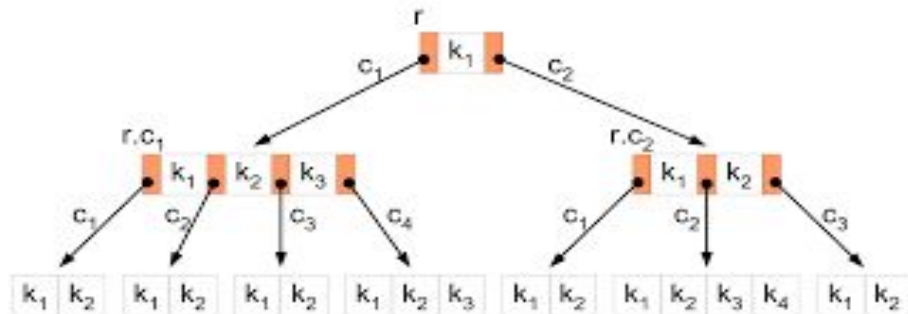
Árvore B como solução

Nó e fator de
ramificação grandes



Características de uma Árvore B

- Árvore auto balanceada.
- Armazena várias chaves por nó.
- Chaves ordenadas.
- Número de filho = número de filhos + 1.
- Filhos separados de modo que as chaves de posição respectiva sejam os fatores de separação
- Todas as folhas têm a mesma nível.
- $h \leq \log_t (n+1/2)$



Operação de Busca em uma Árvore B

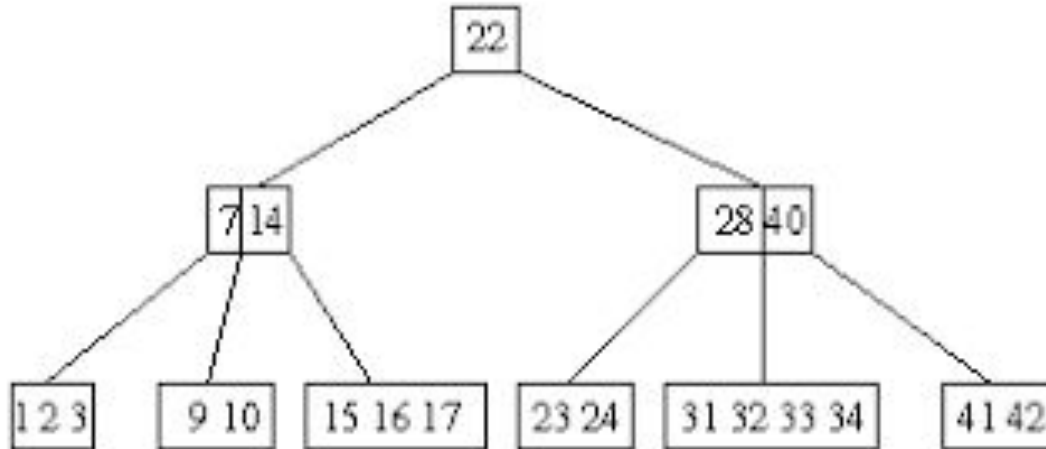
Busca_Arvore_B(x, k)

```
1   i = 1
2   while i <= x.n e k > x.chave[ i - 1]
3       i = i + 1
4   if i <= x.n e k = x.chave[ i - 1]
5       return (x, i)
6   else if x.folha
7       return NULL
8   else Disk-Read(x.filho[ i - 1])
9   return Busca_Arvore_B(x.filho[ i - 1], k)
```

- Comparação das chaves dentro do nó.
- Escolha do filho adequado.
- Esse processo se repete até chegar em um nó folha.
- O x é o nó, k é a chave que está sendo buscada

Operação de Inserção em uma Árvore B

- Inserção possui uma operação de dividir um nó (split).
- Procurar nó adequado.
- Divide os nós cheios pelo caminho até o nó adequado.



Operação de Inserção em uma Árvore B

Inserção_Na_Arvore_B(T, k)

1 r = T.raiz

2 if r.n == 2t - 1

3 s = Alocar_No()

4 T.raiz = s

5 s.folha = false

6 s.n = 0

7 s.filhos[1] = r

8 Dividir_No_Arvore_B(s, 1)

9 Inserção_Nao_Cheio_Na_Arvore_B(s, k)

10 else Inserção_Nao_Cheio_Na_Arvore_B(r, k)

- Consiste em duas operações de inserção diferentes: Inserção_Na_Arvore_B e Inserção_Nao_Cheio_Na_Arvore_B, e as duas usam a operação de dividir
- T é a árvore em que a chave k está sendo inserida

Operação de Inserção em uma Árvore B

Inserção_Nao_Cheio_Na_Arvore_B(x, k)

```
1   i = x.n
2   if x.folha
3       while i >= 1 e k < x.chave[ i ]
4           x.chave[ i + 1 ] = x.chave[ i ]
5           i = i - 1
6       x.chave[ i + 1 ] = k
7       x.n = x.n + 1
8       Disk-Write(x)
9   else while i >= 1 e k < x.chave[ i ]
10       i = i - 1
11       i = i + 1
12       Disk-Read(x.filhos[ i ])
13       if x.filhos[ i ].n = 2t - 1
14           Dividir_No_Arvore_B(x, i)
15           if k > x.chave[ i ]
16               i = i + 1
17       Inserção_Nao_Cheio_Na_Arvore_B(x.filhos[ i ], k)
```

- Consiste em duas operações de inserção diferentes, e as duas usam a operação de dividir
- O x é o nó em que a chave k está sendo inserida

Operação de Divisão em uma Árvore B

Dividir_No_Arvore_B(x, i)

```
1  z = Alocar_No()
2  y = x.filho[ i ]
3  z.folha = y.folha
4  z.n = t - 1
5  for j = 1 to t - 1
6      z.chave[ j ] = y.chave[ j + t ]
7  if not y.folha
8      for j = 1 to t
9          z.filho[ j ] = y.filho[ j + t ]
10 y.n = t - 1
```

- Dividi um nó em dois e joga sua chave mediana para dentro do nó pai
- É a única forma da árvore crescer.
- Se o nó a ser dividido é a raiz da árvore, cria-se uma nova raiz e dividi a antiga

Operação de Divisão em uma Árvore B

```
11  for j = x.n + 1 downto i + 1
12      x.filho[ j + 1 ] = x.filho[ j ]
13  x.filho[ i + 1 ] = z
14  for j = x.n downto i
15      x.chave[ j + 1 ] = x.chave[ j ]
16  x.chave[ i ] = y.chave[ t ]
17  x.n = x.n + 1
18  Disk-Write(y)
19  Disk-Write(z)
20  Disk-Write(x)
```

- Dividi um nó em dois e joga sua chave mediana para dentro do nó pai
- Se o nó a ser dividido é a raiz da árvore, cria-se uma nova raiz e dividi a antiga

Operação de Remoção em uma Árvore B

- Na remoção temos alguns casos que podem acontecer.
- Remoção em folha.
- Remoção em nó interno.
- Redistribuição (rotação).
- Fusão (merge).

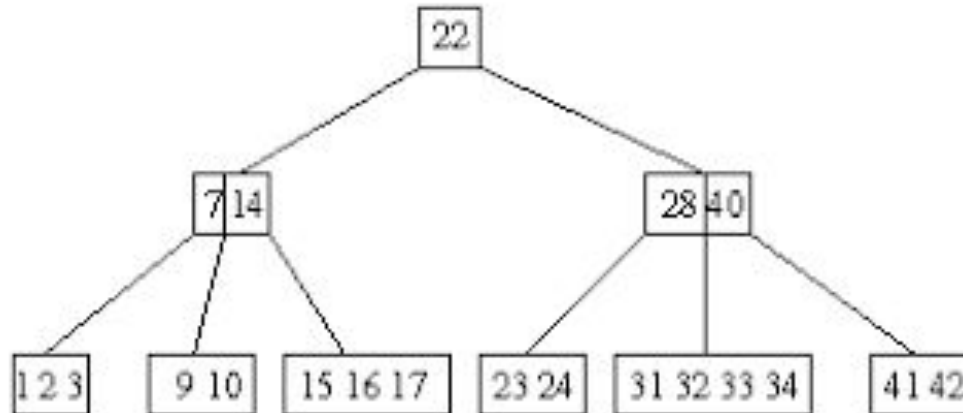


Tabela de Complexidade das Operações na Árvore B

Operações	Complexidade
Busca	$O(t \log t n)$
Inserção	$O(t \log t n)$
Remoção	$O(t \log t n)$

Aplicações da Árvore B

Banco de Dados

Uma das aplicações mais comuns para árvore B.

Gerenciador de Arquivos

Muito usado para organizar diretórios e arquivos.

Exercícios

- 1: Se a raiz de uma árvore B de grau mínimo 3 sofrer uma divisão (split), quantas chaves a nova raiz possuirá logo após a divisão?
- 2: Determine qual é a altura máxima possível para uma árvore B que armazena $n = 50$ chaves, sabendo que seu grau mínimo é $t = 3$.
- 3: Em uma árvore B com $t = 3$, um nó interno possui o pai com a chave $[30, 70]$. Esse pai aponta para dois filhos: o filho esquerdo $A = [10, 20]$ e o filho direito $B = [40, 50, 60]$. O algoritmo solicita a remoção da chave 10 que está no nó A . Como a árvore se reestrutura?